Draw A Game Board ♪♪

Exercise 24 (and Solution)

This exercise is Part 1 of 4 of the Tic Tac Toe exercise series. The other exercises are: Part 2, Part 3, and Part 4.

Time for some fake graphics! Let's say we want to draw game boards that look like this:

```
 --- --- ---
|   |   |   |
 --- --- ---
|   |   |   |
 --- --- ---
|   |   |   |
 --- --- ---
```

This one is 3x3 (like in tic tac toe). Obviously, they come in many other sizes (8x8 for chess, 19x19 for Go, and many more).

Ask the user what size game board they want to draw, and draw it for them to the screen using Python's `print` statement.

Remember that in Python 3, printing to the screen is accomplished by

```
print("Thing to show on screen")
```

Hint: this requires some use of functions, as were discussed previously on this blog and elsewhere on the Internet, like this TutorialsPoint link.

Check Tic Tac Toe

Exercise 26 (and Solution)

This exercise is Part 2 of 4 of the Tic Tac Toe exercise series. The other exercises are: Part 1, Part 3, and Part 4.

As you may have guessed, we are trying to build up to a full tic-tac-toe board. However, this is significantly more than half an hour of coding, so we're doing it in pieces.

Today, we will simply focus on checking whether someone has WON a game of Tic Tac Toe, not worrying about how the moves were made.

If a game of Tic Tac Toe is represented as a list of lists, like so:

```
game = [[1, 2, 0],
        [2, 1, 0],
        [2, 1, 1]]
```

where a `0` means an empty square, a 1 means that player 1 put their token in that space, and a 2 means that player 2 put their token in that space.

Your task this week: given a 3 by 3 list of lists that represents a Tic Tac Toe game board, tell me whether anyone has won, and tell me which player won, if any. A Tic Tac Toe win is 3 in a row - either in a row, a column, or a diagonal. Don't worry about the case where TWO people have won - assume that in every board there will only be one winner.

Here are some more examples to work with:

```
winner_is_2 = [[2, 2, 0],
        [2, 1, 0],
        [2, 1, 1]]

winner_is_1 = [[1, 2, 0],
        [2, 1, 0],
        [2, 1, 1]]

winner_is_also_1 = [[0, 1, 0],
        [2, 1, 0],
        [2, 1, 1]]

no_winner = [[1, 2, 0],
        [2, 1, 0],
        [2, 1, 2]]

also_no_winner = [[1, 2, 0],
        [2, 1, 0],
        [2, 1, 0]]
```

Tic Tac Toe Draw

Exercise 27 (and Solution)

This exercise is Part 3 of 4 of the Tic Tac Toe exercise series. The other exercises are: Part 1, Part 2, and Part 4.

In a previous exercise we explored the idea of using a list of lists as a "data structure" to store information about a tic tac toe game. In a tic tac toe game, the "game server" needs to know where the Xs and Os are in the board, to know whether player 1 or player 2 (or whoever is X and O won).

There has also been an exercise about drawing the actual tic tac toe gameboard using text characters.

The next logical step is to deal with handling user input. When a player (say player 1, who is X) wants to place an X on the screen, they can't just click on a terminal. So we are going to approximate this clicking simply by asking the user for a coordinate of where they want to place their piece.

As a reminder, our tic tac toe game is really a list of lists. The game starts out with an empty game board like this:

```
game = [[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]]
```

The computer asks Player 1 (X) what their move is (in the format `row,col`), and say they type `1,3`. Then the game would print out

```
game = [[0, 0, X],
        [0, 0, 0],
        [0, 0, 0]]
```

And ask Player 2 for their move, printing an O in that place.

Things to note:

- For this exercise, assume that player 1 (the first player to move) will always be X and player 2 (the second player) will always be O.
- Notice how in the example I gave coordinates for where I want to move starting from (1, 1) instead of (0, 0). To people who don't program, starting to count at 0 is a strange concept, so it is better for the user experience if the row counts and column counts start at 1. This is not required, but whichever way you choose to implement this, it should be explained to the player.
- Ask the user to enter coordinates in the form "row,col" - a number, then a comma, then a number. Then you can use your Python skills to figure out which row and column they want their piece to be in.
- Don't worry about checking whether someone won the game, but if a player tries to put a piece in a game position where there already is another piece, do not allow the piece to go there.

Bonus:

- For the "standard" exercise, don't worry about "ending" the game - no need to keep track of how many squares are full. In a bonus version, keep track of how many squares are full and automatically stop asking for moves when there are no more valid moves.

Tic Tac Toe Game 🎵🎵🎵

Exercise 29 (and Solution)

This exercise is Part 4 of 4 of the Tic Tac Toe exercise series. The other exercises are: Part 1, Part 2, and Part 3.

In 3 previous exercises, we built up a few components needed to build a Tic Tac Toe game in Python:

1. Draw the Tic Tac Toe game board
2. Checking whether a game board has a winner
3. Handle a player move from user input

The next step is to put all these three components together to make a two-player Tic Tac Toe game! Your challenge in this exercise is to use the functions from those previous exercises all together in the same program to make a two-player game that you can play with a friend. There are a lot of choices you will have to make when completing this exercise, so you can go as far or as little as you want with it.

Here are a few things to keep in mind:

- You should keep track of who won - if there is a winner, show a congratulatory message on the screen.
- If there are no more moves left, don't ask for the next player's move!

As a bonus, you can ask the players if they want to play again and keep a running tally of who won more - Player 1 or Player 2.